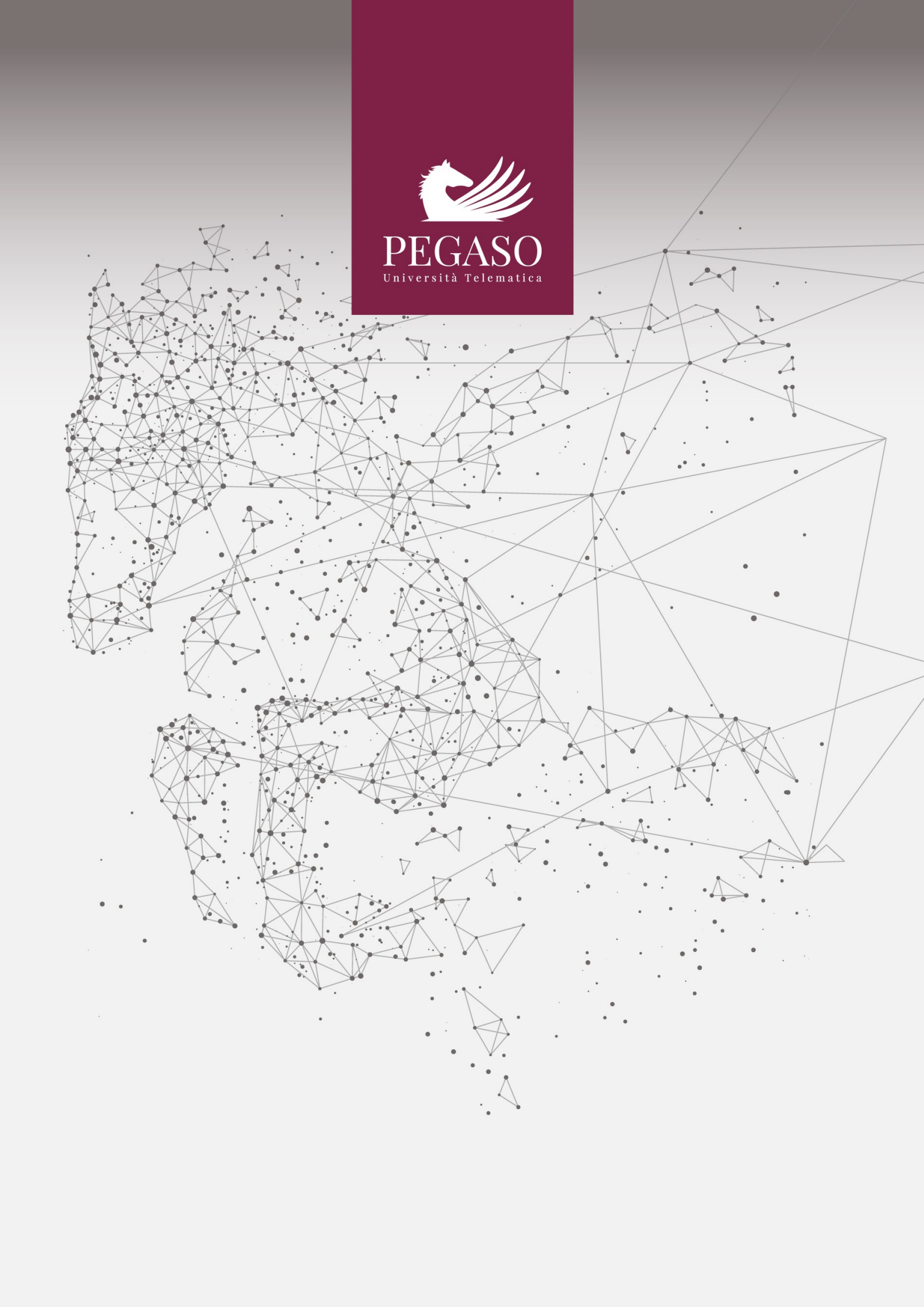

PEGASO
Università Telematica

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. INTRODUZIONE 3
2. CLIENT MULTI-THREAD 4
3. SERVER MULTI-THREAD 6
BIBLIOGRAFIA 9

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Introduzione

I thread hanno una serie di proprietà utili, una delle più importanti delle quali è che possono rendere possibile l'utilizzo di chiamate di sistema bloccanti mentre contemporaneamente si impedisce che l'intero processo in cui il thread sta eseguendo diventi bloccato a sua volta. A causa di questa proprietà, l'utilizzo dei thread nei sistemi distribuiti è particolarmente interessante, poiché rende molto più semplice esprimere la comunicazione sotto forma di mantenimento di più connessioni logiche contemporaneamente. Ciò rende i thread particolarmente attraenti per l'utilizzo nei sistemi distribuiti. Per dimostrare questo punto, in questa lezione esamineremo in modo più approfondito i client e i server multi-thread.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Client multi-thread

I sistemi distribuiti che operano in reti geografiche potrebbero dover nascondere lunghi tempi di propagazione di comunicazione tra processi per ottenere un elevato grado di trasparenza della distribuzione. In una rete WAN (Wide Area Network), la latenza di andata e ritorno può facilmente essere dell'ordine di centinaia di millisecondi e può persino raggiungere occasionalmente secondi.

È pratica comune iniziare una comunicazione e poi passare immediatamente a qualcos'altro come mezzo per nascondere un ritardo nel processo di comunicazione. I browser Web offrono un classico esempio di questo fenomeno in azione. Un documento Web è in genere costituito da un file HTML contenente testo normale, nonché da una raccolta di immagini, icone e altri tipi di supporti. Il browser deve stabilire una connessione TCP/IP, leggere i dati man mano che arrivano e quindi inviarli a un componente di visualizzazione per recuperare ogni singolo elemento di un documento Web. Sia la creazione di una connessione che la lettura dei dati ricevuti sono esempi di operazioni che sono intrinsecamente bloccanti. Quando si tratta di comunicazioni a lunga distanza, ci troviamo di fronte all'ulteriore sfida di dover potenzialmente attendere una notevole quantità di tempo per il completamento di ogni singola operazione.

Un passaggio tipico nel funzionamento di un browser Web consiste nel recuperare la pagina HTML prima di procedere alla visualizzazione. Alcuni browser iniziano a visualizzare i dati mentre sono ancora in arrivo, il che viene fatto nel tentativo di nascondere il più possibile le latenze di comunicazione. Mentre il testo viene reso disponibile all'utente, il browser continua a recuperare altri file che compongono la pagina, come le immagini, ciò che include le funzionalità per lo scorrimento e altre funzionalità simili. Questi ultimi vengono presentati al pubblico man mano che vengono introdotti. L'utente non deve attendere che tutti i componenti della pagina, inclusa l'intera pagina stessa, siano stati recuperati prima che la pagina possa essere resa disponibile.

In effetti, si può osservare che il browser Web sta lavorando su più task contemporaneamente. È stato scoperto che lo sviluppo del browser sotto forma di client multi-thread rende le cose significativamente più semplici. Quando il file HTML primario è stato ottenuto, è possibile attivare thread separati per recuperare gli altri file componenti. Questi thread saranno responsabili di ottenere tali file. Ogni singolo thread crea la propria connessione univoca al server per recuperare i dati. Supponendo che una chiamata bloccante non interrompa l'intero processo, la programmazione di una connessione al server e la lettura dei dati da esso possono essere eseguite utilizzando le chiamate di sistema standard. Il codice per ogni thread è esattamente lo stesso e, soprattutto, è semplice. Nel frattempo, l'utente osserverà solo

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

ritardi nella visualizzazione di immagini e altri elementi, ma sarà in grado di navigare attraverso il documento altrimenti.

È possibile aprire più connessioni contemporaneamente quando si utilizza un browser Web multi-thread, che è un altro vantaggio significativo dell'utilizzo di tale browser. L'esempio precedente è un caso in cui sono state stabilite diverse connessioni allo stesso server contemporaneamente. Se quel server è già molto occupato o è semplicemente lento, non si percepisce alcun miglioramento reale delle prestazioni nel recuperare i file che compongono la pagina rigorosamente uno dopo l'altro. Questo perché estrarre i file uno dopo l'altro è il metodo tradizionale.

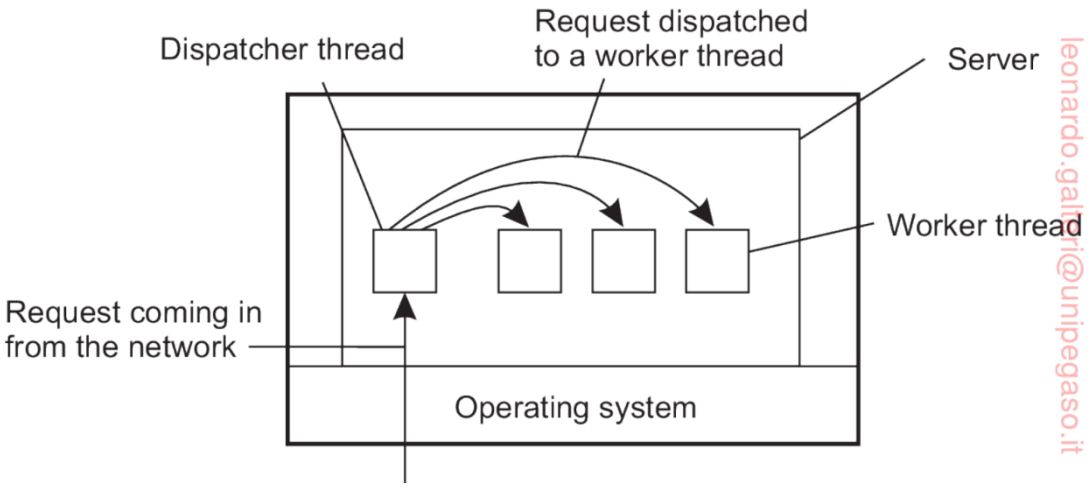
D'altra parte, in molti casi, i server Web sono stati replicati su più macchine e, di conseguenza, ogni server offre esattamente la stessa raccolta di documenti Web. I server replicati condividono la stessa posizione fisica dei server primari e vengono indicati con lo stesso nome. Quando arriva una richiesta per una pagina Web, tale richiesta viene inviata a uno dei server. Questo viene in genere fatto utilizzando una strategia round-robin o qualche altro tipo di tecnica di bilanciamento del carico. È possibile stabilire connessioni a più repliche quando si utilizza un client multi-thread. Ciò consente di trasferire i dati in parallelo, il che alla fine comporta la visualizzazione dell'intero documento Web in un lasso di tempo significativamente più breve rispetto a quando si utilizza un server che non replica i dati. Ad ogni modo, solo se il cliente è in grado di gestire flussi paralleli di dati in entrata questa strategia può essere implementata con successo. I thread sono uno strumento estremamente efficace per raggiungere questo obiettivo.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Server multi-thread

L'applicazione principale del multithreading nei sistemi distribuiti si trova sul lato server, nonostante il fatto che i client multi-thread abbiano una serie di importanti vantaggi. È stato dimostrato attraverso l'applicazione pratica che il multithreading non solo semplifica notevolmente il codice del server, ma rende anche molto più semplice sviluppare server che utilizzano il parallelismo per ottenere prestazioni elevate, e questo è vero anche su sistemi a singolo processore. Tuttavia, nel mondo odierno dei processori multicore, il multitasking per ottenere il parallelismo è la strada più naturale da intraprendere.



leonardo.galteri@unipegaso.it

Figura 1 - Courtesy of van Steen, Maarten & Tanenbaum, Andrew. (2017). Distributed Systems, Third edition

Si consideri l'organizzazione di un file server che, a volte, deve bloccarsi mentre attende che il disco diventi disponibile, al fine di comprendere i vantaggi dell'utilizzo dei thread durante la scrittura del codice per la programmazione del server. In uno scenario tipico, il file server attenderà una richiesta per un'operazione sui file prima di eseguire l'azione richiesta, dopodiché invierà la risposta. La figura 1 illustra uno dei tanti layout organizzativi possibili, uno dei quali è abbastanza comune. Un singolo thread, noto come dispatcher, è responsabile della lettura di tutte le richieste in arrivo per le operazioni sui file. Le richieste vengono trasmesse dai client a un endpoint conosciuto per questo server. Dopo aver esaminato la richiesta, il server seleziona un thread di lavoro attualmente inattivo e gli passa la richiesta.

Il worker continua eseguendo una lettura bloccante sul file system locale, che può comportare l'arresto del thread fino a quando i dati non vengono recuperati dal disco. Nel caso in cui il thread venga sospeso, viene scelto il thread successivo nella coda per essere eseguito. Ad esempio, il dispatcher

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

potrebbe essere scelto per eseguire ulteriori task. In alternativa, è possibile scegliere un thread di lavoro diverso che al momento è pronto per funzionare.

Ora si prenda considerazione come il file server avrebbe potuto essere scritto se i thread non fossero stati disponibili. È possibile farlo funzionare come un singolo thread tra le varie opzioni che abbiamo a disposizione. Il ciclo primario del file server accetta una richiesta, esegue alcune analisi su di essa e quindi la elabora fino in fondo prima di passare a quella successiva. Durante il periodo in cui il server è in attesa del disco, rimane inattivo e non elaborerà altre richieste. Come conseguenza diretta di ciò, le richieste di altri clienti non possono essere soddisfatte. Inoltre, se il file server funziona su una macchina dedicata, come spesso accade, la CPU banalmente non esegue nulla mentre attende che il disco diventi disponibile. Il risultato finale è che un numero significativamente inferiore di richieste può essere elaborato all'interno di una determinata unità di tempo. Pertanto, i thread comportano una quantità significativa di prestazioni, nonostante il fatto che ogni thread sia ancora programmato in sequenza, come di consueto.

Sia un file server multi-thread che un file server a thread singolo sono stati presentati come potenziali progetti fino a questo punto. Il funzionamento del server come se fosse una grande macchina a stati finiti a thread singolo è una terza possibile opzione. L'unico thread esaminerà una richiesta ogni volta che ne viene ricevuta una. Se può essere soddisfatto dalla cache in memoria, allora va tutto bene; altrimenti, in caso contrario, il thread dovrà accedere al disco. Tuttavia, invece di eseguire un'operazione di blocco del disco, il thread pianifica un'operazione del disco asincrona (cioè non bloccante) per la quale verrà successivamente interrotta dal sistema operativo. Ciò consentirà al thread di continuare l'elaborazione. Affinché ciò abbia esito positivo, il thread registrerà prima lo stato della richiesta (vale a dire, che ha un'operazione del disco in sospeso), quindi continuerà a controllare se ci sono state altre richieste in arrivo che richiedono la sua attenzione.

Il sistema operativo invierà una notifica al thread non appena un'operazione del disco in sospeso è stata completata. Il thread quindi, dopo un periodo di tempo appropriato, cercherà lo stato della richiesta associata e continuerà a elaborarla. Dopo un certo tempo, verrà infine inviata una risposta al client che ha avviato la comunicazione. Questa volta, verrà utilizzata una chiamata non bloccante per trasmettere il messaggio sulla rete.

Il modello di "processo sequenziale" che avevamo nei primi due casi non è più possibile grazie a questo design. Quando il thread deve eseguire un'operazione bloccante, deve sempre registrare esattamente dove si trovava nel processo di elaborazione della richiesta e potrebbe anche essere necessario archiviare ulteriori informazioni sullo stato. Dopo che è stato completato, sarà possibile iniziare l'operazione e procedere con gli altri lavori. Quando parliamo di "altro lavoro", intendiamo cose come l'elaborazione di richieste appena arrivate o richieste di post-elaborazione per le quali un'operazione

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

precedentemente avviata è ora terminata. Ovviamente, se non c'è lavoro da fare, il thread si bloccherà se non può andare avanti. In pratica, stiamo simulando il comportamento di più thread e gli stack associati a ciascuno di essi nel modo più laborioso possibile. La procedura viene eseguita come se fosse una macchina a stati finiti, che riceve un evento e quindi risponde ad esso in modi diversi a seconda di ciò che l'evento contiene.

Dovrebbe essere ovvio a questo punto quali benefici si possono ottenere dai thread. Consentono di mantenere l'idea di processi sequenziali che bloccano le chiamate di sistema e raggiungono ancora il parallelismo. La programmazione può essere semplificata quando le chiamate di sistema sono bloccate in modo che appaiano invece come chiamate di procedura standard. Inoltre, l'utilizzo di più thread consente il parallelismo, che a sua volta porta a miglioramenti delle prestazioni. La facilità e la semplicità del blocco delle chiamate di sistema sono mantenute dal server a thread singolo, ma le prestazioni possono essere gravemente compromesse in termini di numero di richieste che possono essere gestite per unità di tempo. L'approccio noto come macchine a stati finiti raggiunge alte prestazioni attraverso il parallelismo; Tuttavia, impiega chiamate non bloccanti, che sono notoriamente difficili da programmare e quindi difficili da gestire.

Per ribadire, è importante tenere presente che possiamo organizzare un server utilizzando più processi in alternativa all'utilizzo di thread, il che comporterebbe la situazione in cui avremmo un server multiprocesso. Un vantaggio è che il sistema operativo può fornire un livello più elevato di protezione contro l'accesso non autorizzato ai dati condivisi. Quando si utilizzano processi anziché thread, tuttavia, potremmo osservare una notevole diminuzione delle prestazioni se c'è molta necessità che i processi comunichino tra loro.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- M. van Steen and A.S. Tanenbaum, Distributed Systems, 3rd ed., distributed-systems.net, 2017.
- Stevens W. UNIX Network Programming – Interprocess Communication. Prentice Hall, Englewood Cliffs, N.J., 2nd edition, 1999.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).